Contents

1 Lecture 1 **2**
1.1 Randomized Min Cuts . . . 2
1.2 Improving Min Cut . . . 3

2 Lecture 2 **4**
2.1 Probability Basics . . . 4
2.2 Hashing . . . 5

3 Lecture 3 **6**
3.1 Hashing and Graph Sparsification . . . 6

4 Lecture 4 **7**
4.1 Graph Sparsification, continued . . . 7
4.2 Linear Programming . . . 7
4.3 Basic Applications of Linear Programming . . . 8

5 Lecture 5 **9**
5.1 Linear Programming and Applications, Continued . . . 9
5.2 MAX 2-SAT . . . 9
5.3 Min Makespan Scheduling . . . 9

1 Lecture 1

In this class, we will explore the following themes.

1. Randomness - When the worst-case model means there are not good algorithms, randomized error models can give you better guarantees, in line with reality. Randomized algorithms also can be better (on average) than deterministic ones on worst-case inputs.
2. Optimization - Continuous optimization can help design great algorithms. Examples include linear programming, semidefinite programming, gradient descent, and stochastic gradient descent.
3. Algebraic Techniques - Singular Value Decomposition, low-rank approximation and spectral methods use linear algebra to build useful algorithms. Furthermore, we can try to solve systems of polynomial equations.

Even more important is the interplay between these three themes.

1.1 Randomized Min Cuts

Karger's 1993 paper gave a simple algorithm for finding minimum cuts in graphs. Define a graph G with vertices V and edges E. Here $n = |V|$ and $m = |E|$.

Definition 1.1
Given a graph $G = (V, E)$ and $S \subseteq V$, define the cut $\mathrm{Cut}(S) = E(S, \bar{S})$. Our goal is to find $S \neq \emptyset, V$ that minimizes $|\mathrm{Cut}(S)|$.

Definition 1.2 (Contraction)
The contraction of an edge $e \in E$ produces a new graph $G' = (V', E)$ where $V' = (V \setminus \{u, v\}) \cup S_{u,v}$ (combine u and v into a "super-vertex") and

$$E' = \{\{S_{u,v}, w\} \mid \{u, w\} \text{ or } \{v, w\} \in E\} \cup \{e \in E \mid u \notin e, v \notin e\}$$

where we take the edges that we add as a multigraph (multiple edges between the same pair of vertices).

Notice that every time a contraction is performed, the number of vertices decreases by 1 and the number of edges decreases (by maybe more than one, due to multiedges). Here is the contraction algorithm.

Algorithm 1.3 (Karger's Min Cut)

- $H = G, H = (V', E')$.
- While H has more than 2 vertices:
- Pick a random edge on E', $\{u, v\}$.
- Contract H on $\{u, v\}$.
- Consider the number of edges between the two vertices left as a minimum cut candidate.

Theorem 1.4
Fix a min-cut $(X, \bar{X})$ in G. We claim the algorithm above returns $|\mathrm{Cut}(X)|$ with probability at least $\frac{2}{n(n-1)}$.

Thus, it suffices to run the original algorithm independently t times and take the smallest cut we see. The probability

that we make a mistake all t times is

$$\mathrm{P}\,(\text{failure chance}) \leq \left(1 - \frac{2}{n(n-1)}\right)^t \leq e^{-2\frac{t}{n(n-1)}}$$

if we take $t = O(n^2 \log n)$, then the failure chance is at most $\frac{1}{n^{O(1)}}$. Furthermore, the contractions each take $O(n)$ time and there are $O(n)$ iterations. Thus, the runtime of the whole algorithm is poly-time, taking at most $O(n^4 \log n)$ time. We will soon find a speedup, but first let us prove the theorem.

Proof 1.5
Note that any cut in the contracted graph $\mathrm{Cut}(S)$ in H can be mapped one-to-one to some cut in G. Further, every cut in G that doesn't split a contracted pair of vertices has a corresponding cut in H. Furthermore, a cut survives a step of Karger's if you do not contract one of its edges. The probability that you miss the cut $(X, \bar{X})$ is $1 - \frac{|\mathrm{Cut}(X)|}{|E'|}$. Furthermore, suppose the minimum cut has c edges, then G has at least $\frac{cn}{2}$ edges in total. This is because the degree of each vertex must be at least c. Thus, at every step, the survival probability is at least $1 - \frac{c}{\frac{c|V'|}{2}}$. Thus, overall

$$\mathrm{P}\,(\text{survival of cut}) \geq \prod_{i=3}^{n}\left(1 - \frac{2}{i}\right) = \prod_{i=3}^{n}\left(\frac{i-2}{i}\right) = \frac{(n-2)!}{n!/2!} = \frac{2}{n(n-1)}$$

1.2 Improving Min Cut

We saw that when the number of vertices is large, it's unlikely that we contract the min cut. So, we should not repeat the beginning too many times.

Theorem 1.6
The chance that the minimum cut survives $n - \frac{n}{\sqrt{2}}$ steps of contraction is $\geq \frac{1}{2}$.

Now, we can simply do two independent contractions until there are $\frac{n}{\sqrt{2}}$ vertices in each. Then we can recursively calculate minimum cuts in both and take the better one. Then we can use a recurrence to calculate the running time.

$$T(n) = O(n^2) + 2T\left(\frac{n}{\sqrt{2}}\right) \implies T(n) = O(n^2 \log n)$$

Finally, we should prove the following theorem

Theorem 1.7
The algorithm finds a minimum cut with probability at least $\frac{\Delta}{\log_2 n}$.

Proof 1.8
Call $P(n)$ the chance the algorithm works on a graph of size n. Then $P(n) \geq 1 - (1 - \frac{1}{2}P(n/\sqrt{2}))^2$. By induction, we can prove the claim.

2 Lecture 2

2.1 Probability Basics

We started class with a probability review. Most was generic stuff. But, the following definition is a bit rare.

Definition 2.1
$X_1, \ldots, X_n$ are parwise (k-wise, respectively) independent of every pair (k-tuple, respectively) are independent (their probabilities multiply).

We move the discussion to useful tail bounds to estimate tail probabilities.

Theorem 2.2 (Markov's Bound)
If X is a nonnegative random variable,

$$\mathbb{P}\left[X \geq k\mathbb{E}\left[X\right]\right] \leq 1/k$$

If in addition, $X \leq z$, then

$$\mathbb{P}\left[X \leq c\mathbb{E}\left[X\right]\right] \leq \frac{z - \mathbb{E}\left[X\right]}{z - c\mathbb{E}\left[X\right]}$$

Theorem 2.3 (Chebyshev's Inequality)
For any random variable,

$$\mathbb{P}\left[|X - \mathbb{E}\left[X\right]| \geq k\sigma\right] \leq 1/k^2$$

Recall that a Gaussian distribution $N(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$ concentrates very well around its mean. In particular, if $X \sim N(\mu, \sigma^2)$, $\mathbb{P}\left[|X - \mu| \geq t\sigma\right] \leq O(e^{-t^2/2})$. It turns out that there are other settings that mimic Gaussians (largely due to the central limit theorem),

Theorem 2.4 (Chernoff/Hoeffding's Bound)
(Bernstein's Inequality) Let $X_1, \ldots, X_n$ be independent and they are bounded (say, $X_i \in [-1, 1]$). Call $\mathbb{E}\left[X_i\right] = \mu_i$, $\sigma_i^2 = \mathrm{Var}\left(X\right)_i$ and $X = \sum_i X_i$. Suppose $k \leq \frac{\sigma}{2}$.

$$\mathbb{P}\left[|X - \mu| \geq k\sigma\right] \leq 2\exp\left(-\frac{k^2}{4}\right)$$

(Chernoff Bound) Let $X_1, \ldots, X_n \in \{0, 1\}$ be independent and $p_i = \mathbb{E}\left[X_i\right]$. Then,

$$\mathbb{P}\left[\sum_i X_i \geq (1 + \epsilon)\mu\right] \leq e^{-\frac{\epsilon^2\mu}{3+\epsilon}}$$

Proof 2.5
We will use the so-called moment method. Define $Y = e^{tX}$ and

$$\begin{aligned}\mathbb{E}[Y] &= \mathbb{E}\left[e^{t\sum_i X_i}\right] = \mathbb{E}\left[\prod_i e^{tX_i}\right] = \prod_i \mathbb{E}\left[e^{tX_i}\right] \\ &= \prod_i (p_i e^t + (1-p_i)) = \prod_i (1 + p_i(e^t - 1)) \leq \prod_i e^{p_i(e^t-1)} = e^{\mu(e^t-1)}\end{aligned}$$

Thus, by Markov's,

$$\mathbb{P}[X \geq (1+\epsilon)\mu] = \mathbb{P}\left[Y \geq e^{t\mu(1+\epsilon)}\right] \leq \frac{e^{\mu(e^t-1)}}{e^{\mu t(1+\epsilon)}}$$

Optimizing t, as $t = \log(1+\epsilon)$ and using $\log(1+\epsilon) \approxeq \epsilon - \frac{\epsilon^2}{2}$, we get the bound.

2.2 Hashing

We have a universe U and we wish to map it to a small set of size $n \ll |U|$ using a function h. The data structures we want to support are insert(x), delete(x) and search(x), which have self explanatory behavior. We can use randomness to make our data structure to be robust against adversarial behavior.

A simple idea is to make h a function a which maps each element as it arrives into an independent random slot. If there are $m = O(n)$ elements that come in, the load on any slot is $\leq O(\log n)$ with high probability.

Theorem 2.6
The load on any slot is at most constant with high probability for $m < n$.

We can start the proof as follows. Let L_x be the number of elements in the same slot as x. Define I_y be the indicator that $h(y) = h(x)$. It's clear that $L_x = 1 + \sum_{y \neq x} I_y$. Thus, $\mathbb{E}[L_x] = 1 + \sum_{y \neq x} \mathbb{E}\left[I_y\right] = 1 + \frac{m-1}{n} \leq 2$ if $m - 1 \leq n$. We will pause here to note that if we reduce the amount of randomness, then we can get the same result. Namely, let $\mathcal{H}$ be a family of hash functions and h is a uniformly random element of $\mathcal{H}$. Namely, as long as the values of h are pairwise independent, $\mathbb{E}\left[I_y\right] = \frac{1}{n}$ (in fact, this is an even weaker notion of pairwise independence).

3 Lecture 3

3.1 Hashing and Graph Sparsification

ZZZ...

4 Lecture 4

4.1 Graph Sparsification, continued

Recall that last time we wanted to find an ϵ-cut sparsifier for a graph and we proposed the following algorithm.

Algorithm 4.1
Keep every edge independently with $p = c\frac{\log n}{k\epsilon^2}$ where k is the size of the min cut in G. If you keep an edge, make its weight $1/p$. Then this is a ϵ-cut sparsifier.

Proof
First, let's try a heuristic calculation. Consider a cut $(S, \overline{S})$. The weight of edges in the cut that are left after the process is a $p\cdot$ a binomial random variable, whose expectation is exactly $\frac{|\mathrm{Cut}(S)|}{p}$. Thus, in expectation the cut stays roughly the same. The problem is, our concentration bounds don't quite guarantee that this happens for all cuts. The probability that we deviate more than a $(1 \pm \epsilon)$ factor, by the Chernoff bound, is $2e^{-O(p\mathrm{Cut}_G(S)\epsilon^2)}$. This expression is the largest when the cut is the smallest, i.e. the min-cut, but would only give a probability of $1/n^{O(1)}$. However, there are exponentially many cuts, so taking a naive union bound will not work.
We thus use the following fact which will be proved in homework:

Fact 4.2
The number of α-approximate min cuts is at most $n^{2\alpha}$.

That is, the number of cuts with size at most αk is at most $n^{2\alpha}$. Now, we can bucket the cuts by their size and union bound over them individually, a so-called **stratified union bound**. Let's have each bucket i contain the cuts of size $[2^{i-1}k, 2^i k)$. Now, in this bucket the number edges is outside of $(1 \pm \epsilon)$ with probability at most $e^{-c2^{i-1}\log n}$ and there are only at most $n^{2^{i+1}}$ such cuts, so it holds for any of them with probability at most $e^{-c(2^{i-1}+2^{i+1})\log n} = e^{-O(2^{i-1}\log n)} = n^{-O(2^{i-1})}$. Now, summing up this probability over the buckets will give a series, which is about $\frac{1}{n^c} + \frac{1}{n^{2c}} + \frac{1}{n^{4c}} + \ldots$, whose sum is bounded by a constant times the first term. Thus, with high probability, all cuts are sparsified.

4.2 Linear Programming

What's the runtime of Gaussian eliminiation? Some may say that it terminates in $O(n^3)$ steps, but notice that each step is really $O(1)$ "field operations." If you're over a small finite field, these are fast, but over the reals you could have to do a really precise operation that takes a long time. It turns out, Gaussian elimination DOES run in polynomial time (the number of bits in the input does not blow up too much) but this is a very nontrival fact to prove, first proved by Bareiss hundreds of years after the invention of the algorithm.

A natural counterexample to our intuition is the following problem. Given two polygons, both given as sets of rational numbers that represent their vertices, find the one with bigger perimeter. The problem is inherently that the square root cannot be represented accurately without knowing a priori what error margin to round to. However, one cannot be sure what the error margin is unless you're given the difference in the perimeters to begin with!

Another such miracle is that linear programming over the reals runs in polynomial time.

Definition 4.3
Let $a_i \in \mathbb{R}^n$, $b_i \in \mathbb{R}$ for $1 \le i \le m$ and $c_j \in \mathbb{R}$ for $1 \le j \le n$. Then a linear program is the following minimization problem

$$\min_{x_j \in \mathbb{R}} \sum_{j=1}^{n} c_j x_j : \{a_i^\top x \le b_i \quad \forall 1 \le i \le m\}$$

Let a_i be the rows of some matrix A and stack the b_i into a vector b. Then we can rewrite the constraints as $Ax \le b$.

It turns out linear programming is a very applicable and extensible framework for solving problems.

4.3 Basic Applications of Linear Programming

One simple problem is an allocation setting where you have n jobs and n factories. Each job is assigned to precisely one factory and assigned job i to factory j has a cost $p_{ij} \geq 0$. You wish to minimize the cost of all jobs in aggregate. Let x_{ij} be 1 if job i is assigned to factory j and 0 if not. Then the following integer program exactly captures the problem.

$$\min \sum_{i,j} p_{ij} x_{ij} : \begin{cases} x_{ij} \in \{0,1\} & \\ \sum_i x_{ij} = 1 & \forall j \\ \sum_j x_{ij} = 1 & \forall i \end{cases}$$

The second and third constraints that every job and every factory have exactly one match. However, since integer programming is NP-Hard, we instead **relax** the integrality constraint to instead just allow $0 \leq x_{ij} \leq 1$. This means the set of feasible solutions is larger, so the minimization value of the new LP could be smaller.

For this LP, the problem is now the solution we get may be fractional, where the x_{ij} are not integers, and so we would not immediately get an allocation. However, it is possible to prove that the optimal value achieved by this LP is actually EQUAL to the integer program's value. This is not usually true, we actually got lucky (something called **total unimodularity** was present here).

Let's look at a different problem where LPs won't immediately get you the optimal solution. Consider the problem of maximum weighted vertex cover.

Definition 4.4
The maximum weighted vertex cover (MWVC) problem is a problem where given a graph $G = (V, E)$ and a function $w : V \to \mathbb{R}_+$, we wish to find a set of vertices S such that all edges are touched by at least one vertex in S where $\sum_{v \in S} w(v)$ is minimized.

Now, this problem is NP-Hard. Nevertheless, we can write an IP that represents the situation and relax it into an LP:

$$\min \sum_i w(i) x_i : \begin{cases} 0 \leq x_i \leq 1 & \forall i \\ 1 \leq x_i + x_j & \forall (i,j) \in E \end{cases}$$

where x_i is meant to be 1 if we take vertex i into the cover and 0 if not, and the second constraint makes sure that we cover all the edges. Now, given a fractional solution, we will instead **round** the answer to get a set of vertices to pick. In this particular instance, we will take vertex i if $x_i \geq 0.5$, otherwise we will not. This will indeed cover every edge; for each edge (i,j), $x_i + x_j \geq 1$ so one of them is at least 0.5. However, we might take more vertices than we need to; indeed we claim that our vertex cover has weight $2 \cdot OPT$, making it a 2-approximation. To see this, notice that for the LP solution $\sum_i w(i) x_i \leq OPT$. However, each x_i at most doubles (ones rounded to 1 are at most doubled and ones rounded to 0 are zeroed), so therefore the returned cover has weight at most $2(\sum_i w(i) x_i) \leq 2 \cdot OPT$ as claimed.

Assuming the complexity theory conjecture that Unique Games is NP-Hard, then this LP is actually the best approximation we can do for the problem! So far nothing better is known.

5 Lecture 5

5.1 Linear Programming and Applications, Continued

Last time, we relaxed a discrete problem (weighted vertex cover) into a continuous linear program, then rounded that linear program according to a deterministic rounding scheme. We can also use a "randomized rounding scheme."

5.2 MAX 2-SAT

Consider the problem of **MAX 2-SAT**.

Definition 5.1
A **MAX 2-SAT** instance is given by n variables $x_1, \ldots, x_n$ and m clauses with two literals (a variable or its negation) in the form $(\overline{x_i} \vee x_j)$, we wish to figure out the maximum number of clauses that can be satisfied by an assignment to the variables to $\{0, 1\}$ (Booleans).

Note that the related satisfiability problem **2-SAT** is solvable in polynomial time, wherein we only wish to know whether ALL clauses are satisfiable or not. The above problem is NP-Hard to approximate to a ratio of better than 0.9546. Notice that a random assignment achieves a $3/4$ approximation in expectation. Let's try to formulate this as a linear program.

$$\max \sum_c z_c : \begin{cases} 0 \leq x_i \leq 1 & \forall i \\ z_c \leq y_{c_1} + y_{c_2} & \forall c \end{cases}$$

where $y_{c_j} = x_{c_j}$ if x is not negated and $y_{c_j} = 1 - x_{c_j}$ if x is negated. Our LP solution will get a fractional solution and we will round $v_i = 1$ with probability x_i. If we have a 1-clause, then we do better than LP-OPT in expectation, as the probability we satisfy it is at least $x_{c_j} \geq z_c$. For every clause c, $\mathbb{E}\left[[\text{] rounding}\right] \geq \frac{3}{4} z_c$. The probability that a clause c is rounded to 1 is $1 - Pr(\text{both are zero}) = 1 - (1 - y_{c_1})(1 - y_{c_2})$. Expanding, we get that

$$y_{c_1} + y_{c_2} - y_{c_1} y_{c_2} = y_{c_1} + y_{c_2} - \frac{[(x_{c_1} + x_{c_2})^2 - (x_{c_1} - x_{c_2})^2]}{4} \geq (x_{c_1} + x_{c_2}) - \frac{(x_{c_1} + x_{c_2})^2}{4}$$

If the sum is at most 1, then its at most $\frac{3}{4}(x_{c_1} + x_{c_2})$. Otherwise, the minimum of the quadratic is still at most $3/4$.

5.3 Min Makespan Scheduling

We have n jobs and m machines and $p_{i,j}$ is the job completion time if i is assigned to j. The goal is to minimize the total job completion time under a "make span" model, with infinitely many parallelism. We can state the objective as

$$\min \max_j \sum_i x_{i,j} p_{i,j} : \{x_{i,j} \in \{0, 1\}\}$$

To linearize the constraints, we can just write

$$\min T : \begin{cases} x_{i,j} \in [0, 1], \sum_j x_{i,j} \geq 1 & \forall i \\ T \geq \sum_i p_{i,j} x_{i,j} & \forall j \end{cases}$$

However, this LP has a very bad integrality gap; for instance if there is just 1 job and all $\{p_{1,j}\}$ are 1, then the LP opt is only $1/m$. Consider parameterizing the LP by t

$$\min T : \begin{cases} x_{i,j} \in [0, 1], \sum_j x_{i,j} \geq 1 & \forall i \\ T \geq \sum_i p_{i,j} x_{i,j} & \forall j \\ x_{i,j} = 0 & \forall i, j \, p_{i,j} \geq t \end{cases}$$

For an optimal $t = T^*$, clearly the integral solution is feasible. Since there are a polynomial number of t's, we can brute force over t and run that many LPs. Our goal is then to find an integral schedule with makespan at most $T + t$. This yields a 2-approximation to the makespan.

Algorithm 5.2
We will round the above LP as follows.

1. For each j, $w_j = \lceil \sum_i x_{i,j} \rceil$.
2. Create a bipartite graph $G = (L, R, E)$ with $L = [n]$ and R as w_j copies of each machine j, labeled as $j_1, \ldots, j_{w_j}$.
3. For each j, we will sort all jobs by descending $p_{i,j}$, so $p_{(1)j} \geq p_{(2)j} \geq \cdots \geq p_{(n)j}$.
4. We will allocate $x_{(1)j}$ to j_1, $x_{(2)j}$ to j_1, and so on until we "fill" up the capacity to 1. By that we mean we add an edge $\{(i), j_1\}$ with weight $x_{(i)j}$.
5. Perform maximum bipartite matching on G to find an integral assignment of the same cost. (This is a flow theorem.)

Proof
Now what is the total makespan? Well the jobs in the integral assignment assigned to j_{c-1} is at least the cost of any job assigned to j_c. Let T_c be the processing time of the slowest job assigned to j_c. Then the makespan at j is at most $\sum_c T_c$. Note that $\sum_{i,j} x_{i,j} p_{i,j} \geq \sum_{c=2}^{w_j} T_c$, because if job i is assigned to copy j_c, then $p_{i,j} \geq T_{c+1}$. Furthermore $t \geq T_1$. Thus, $\sum_c T_c \leq T + t$, as required.